

MediPilot: Implementation README

Architecture, Data, Training, Evaluation, Execution



Aditya Onam Aditya Gupta Varada Patel

Team 01 BIT • Indian Institute of Technology Patna • August 2026 • branch restructure

ABSTRACT

This document is the implementation README for the MediPilot prototype: what was built, how the tree is arranged, what it runs on, how its data is manufactured, how the model is trained and evaluated, and the exact commands that drive each part. The white paper argued a clinical contract — *the system may raise a waiting patient's priority autonomously and may never lower it below a human-assigned level*. This document is the account of how that contract survives contact with code. Three claims organise it. First, the architecture is split into three tracks that cannot borrow each other's authority: a perception track that may only *report*, an interface track that may only *render*, and an evaluation track that alone may *decide*. Second, every model in the system has a deterministic layer beneath it, so a missing API key, an absent GPU or a schema violation degrades the demonstration rather than stopping it. Third, the clinical behaviour is configuration, not code: six YAML files under `backend/config/` hold the strata, cadences, red flags, surge policy, label spec and the feature-leakage registry, and the registry fails the build closed on any unregistered field. We give the directory structure, the current stack, the runtime workflow end to end, the synthetic-data generator and the three structural choices that stop its labels being circular, the training and evaluation pipelines with their committed numbers, a usage guide, the stack this would move to under load, and a plainly stated verification position — 278 passing tests, 4 known failures, each named and diagnosed.

Scope: directory structure • current and target tech stack • solution architecture • synthetic data generation • training • evaluation • execution and usage • verification evidence

INDEX

1 Implementation Approach	p. 1	5 Synthetic Data Generation	p. 5	9 Scaling: the Target Stack	p. 9
2 Directory Structure	p. 2	6 Model Training	p. 5	10 Verification & Known Issues	p. 10
3 Tech Stack & Dependencies	p. 2	7 Evaluation	p. 6		
4 Solution Architecture	p. 3	8 Using This Repository	p. 8		

1 IMPLEMENTATION APPROACH

The prototype was not built as one application with a model bolted into it. It was built as three tracks with a deliberate authority gradient between them, because the failure this project exists to prevent — a patient deteriorating unobserved in a queue — is a failure of *who is allowed to decide what*, not a failure of average accuracy.

The language model may report. The interface may render. Only the deterministic engine may decide. No layer may acquire an authority it was not given, and every layer has a non-model fallback beneath it.

All data in this system is synthetic. No clinical, performance or causal claim is made anywhere in this repository, and `SIMULATED DATA` belongs on every screen. The metrics quoted in §7 describe a model trained on a generator, not on patients.

Six rules follow, and each is visible in the source tree rather than only in the design documents.

1.1 1 — One type contract, mirrored on both sides

The entire frontend/backend boundary is declared once in `web/lib/api/types.ts` and mirrored field-for-field by `backend/triage/orchestrator/dto.py`. A field that does not exist in the DTO cannot reach the nurse board, which makes the contract enforceable by reading two files rather than by auditing the whole surface.

1.2 2 — A tiered fallback ladder, never a single model

Perception is tiered and the bottom tier is always deterministic. Spoken-answer matching runs tier 0 (exact bilingual lexicon

— yes/no and 0–10 in English, Devanagari and romanised Hindi), tier 1 (similarity over labels and synonyms, with an edit-distance rescue for ASR near-misses), and only then tier 2, where a hosted model is asked to pick one of the option values already on the patient's screen. Structuring runs local, then hosted, then rule-based. The order is configuration: `MEDIPILOT_STRUCTURER` and `MEDIPILOT_ASR_BACKEND` each take a comma-separated preference list whose last entry always works offline.

1.3 3 — Clinical behaviour lives in YAML

Six files under `backend/config/` are the clinical surface, and in a real deployment each would be separately signed off. Change behaviour there before changing it in Python (§4).

1.4 4 — The two-clock resource model

The system does not assume continuous monitoring, because Indian EDs do not have it. Facts decay, and the decay is explicit: every measurement carries `{value, takenAt, source, validity}`, and two clocks run per patient — re-score (model) and re-measure (physical), on band-specific cadences from `config/band_cadence.yaml`.

1.5 5 — Offline-first, including the demonstration

`web/lib/api/adapters/mock.ts` runs the entire twenty-patient corpus in the browser, so all seven clinical surfaces are demonstrable with no Python process, no credential and no network. `MEDIPILOT_INTAKE_OFFLINE=1` forces every hosted call to behave as though the network were down — how the kiosk is rehearsed without burning quota, and how it is demonstrated if venue wifi fails.

1.6 6 — Violations are exceptions, not warnings

An attempt to lower a band without an attached human action raises `AsymmetricAutonomyViolation`. Three surge actions — raising the cost ratio to quieten alarms, resolving an abstention by guessing, and de-escalating anyone to free capacity — raise `SurgeViolation`. These are hard errors on the code path, not lint rules or review conventions.

2 DIRECTORY STRUCTURE

The tree was restructured on **2026-08-29**; the work sits on branch `restructure`, cut from `dev`, not yet merged. Before-and-after evidence is under `.migration/`, and every judgement call made during the move is in `.migration/DECISIONS.md`. Git tracks 163 files under `backend/`.

<code>web/</code>	Next.js 16 app, every screen
<code> app/</code>	8 routes incl. <code>/intake /board /audit</code>
<code> api/intake/</code>	server-only Groq handlers (x4)
<code> intake/</code>	kiosk: tree, voice, steps, matching
<code> components/</code>	clinical/ mascot/ 3d/ ui/
<code> lib/</code>	api/ clinical/ voice/ seed/ hooks/
<code> public/media/</code>	poses, video, GLB, textures (~55 MB)
<code>backend/</code>	packages are top-level here
<code> triage/</code>	band engine, audit, surge, api.py
<code> orchestrator/</code>	the /v1 app the web app calls
<code> model/</code>	features, calibration, train, artifacts/
<code> intake/</code>	state machine, tree, structurizer
<code> rules/</code>	red flags, thresholds, spo2 guard
<code> speech/</code>	Groq ASR, Whisper, VAD recorder
<code> data/</code>	synthetic corpus + generator/
<code> eval/</code>	ASR and structurizer bake-offs
<code> config/</code>	the clinical knobs, as YAML
<code> tests/</code>	pytest suite
<code>docs/</code>	paper/ benchmarks/ diagrams/ notes/
	logs/ submission/
<code> experiments/</code>	whisper-live/, not part of the service
<code> archive/</code>	quarantined dead code
<code> .migration/</code>	restructure evidence

2.1 Paths that moved

Any document written before 2026-08-29 — including the root `README.md` — refers to the old tree. The mapping:

Old	New
<code>frontend/</code>	<code>web/</code>
<code>Backend/MediPilot/</code>	<code>backend/</code>
<code>medipilot-model/</code>	
<code>...medipilot-model/</code>	<code>backend/triage/</code>
<code>backend/</code>	
<code>Backend/MediPilot/Metrics/</code>	<code>docs/benchmarks/</code>
<code>Backend/MediPilot/docs/</code>	<code>docs/paper/</code>
<code>.../speech_layer/</code>	<code>experiments/whisper-live/</code>
<code>vigil-diagrams/</code>	<code>docs/diagrams/</code>

Two traps in this tree. A capital-B `Backend/` still appears in `ls` on Windows; it holds nothing tracked, only leftover `__pycache__` and `.pytest_cache` shells that the case-insensitive filesystem merges with the real `backend/`. Delete the cruft or ignore it — do not "fix" it by renaming. Second, about twenty modules resolve config with `Path(__file__).parent.parent / "config"` and one walks up three levels (`triage/orchestrator/seed.py`). Moving a single `.py` file to a different depth breaks it at *runtime*, not at import, so a green collection proves nothing. Move whole subtrees, never lone files.

3 TECHNOLOGY STACK AND DEPENDENCIES

The system is constrained to commodity hardware on purpose. Nothing on a required path needs a GPU, a paid cloud tier

or a database server — because the deployment target is a district hospital, and because DPDP Act 2023 data-locality obligations are easier to honour when the default path never leaves the building.

Runtime	Declared	Verified on this machine
Node.js	20+	v24.16.0
Python	3.10+	3.12.3
GPU	not required	none present; CPU-only throughout
Database	none	state is held in-process
Credential	optional	GROQ_API_KEY, hosted tiers only

3.1 Web (Track B)

Next.js 16.3.2 on the App Router, React 19.2.8, TypeScript in strict mode, Tailwind v4 through `@tailwindcss/postcss`, motion for animation, and `@react-three/fiber` + `three` 0.185 for the kiosk and mascot scenes. Path alias `@/*` resolves to `web/*`. Speech capture is the browser's own `webkitSpeechRecognition` plus the Web Audio API — no speech dependency is bundled. **No test runner is wired up on this side**, which is a real gap rather than an omission from this document.

3.2 Backend (Track C) — a deliberate three-file split

The Python dependencies are split across three files, and the split is a safety boundary rather than tidiness. Only the first is installed by a deployment.

- `requirements.txt` — the serving path: FastAPI 0.111, Uvicorn 0.29, Pydantic 2.7, NumPy 1.26, SciPy 1.13, scikit-learn 1.4.2, PyYAML for the clinical config, joblib for artifacts, python-dateutil, pytest 8.2, httpx, plus groq and soundfile for the hosted perception tiers. scikit-learn is pinned exactly: the artifact manifest records the version it was trained under, and a mismatch is a silent unpickling hazard.
- `requirements-experimental.txt` — `lightgbm` and `xgboost`, for the GBDT bake-off and sequence experiments. Explicitly *not* installed by the API deployment.
- `requirements-speech-local.txt` — the fully local perception path: `faster-whisper` 1.1.1, `soundfile`, `silero-vad`, `sounddevice`, with `openai-whisper` and `torch` kept only for bake-off comparison. It also carries an explicit `requests` pin, because `faster_whisper/utils.py` imports `requests` without declaring it in its own metadata and a clean install otherwise fails at import.

3.3 Perception (Track A)

One credential, `GROQ_API_KEY`, buys both hosted Whisper ASR and the LLM structurizer. Two models, chosen from `docs/benchmarks/structurizer_bakeoff_table.md`: `gpt-oss-120b` (symptom F1 0.962, zero red flags missed) for observation extraction, and `gpt-oss-20b` (F1 0.816) for the cheaper matching and classification calls. A local GPU path was rejected because the development hardware is AMD — no CUDA, no ROCm on Windows — and because the demonstration must not depend on a live tunnel. Decoding `webm/opus` for the pre-flight silence gate needs `ffmpeg` on `PATH`; without it the gate reports itself skipped

rather than silently passing.

3.4 Model

Gradient-boosted trees over 59 features. The shipped artifact is `medipilot-gbdt-v0.2.0` (feature version `fx-v4`, 180-minute horizon, scikit-learn 1.4.2), selected by `model/artifacts/current.txt` and shipped with isotonic calibration, a conformal predictor, per-stratum thresholds and a `manifest.json`. Four alternates — `xgboost`, `hist`, `hist-mono`, `last-obs` — sit beside it for the leaderboard.

4 SOLUTION ARCHITECTURE

MediPilot is three tracks over two runtimes (Fig. 1). The split is physical: Track B is a Node/Next.js payload, Track C is a Python process, and Track A is a set of adapters that either may call but neither may trust.

4.1 Track B — the clinical interfaces

`web/` carries seven clinical surfaces plus a landing page: `/intake` (the patient kiosk), `/board` (the nurse board), `/card/[id]` (the single-patient card and override panel), `/hall` (the waiting-hall display), `/counter`, `/control` (demonstration clock and surge control) and `/audit` (the hash-chained ledger view). `web/components/clinical/` enforces invariants at runtime rather than by convention — `AcuityCard` throws if asked to render a score without a conformal confidence band, and `LockedAcuitySlot` exists so a human-assigned level has a component that structurally cannot be lowered by a machine.

4.2 Track C — the triage engine

`backend/triage/` holds the deterministic `band_engine`, the `surge_controller`, the `recheck_scheduler`, the hash-chained `audit_log` and the orchestrator. `backend/rules/` holds the safety floors: `red_flag_engine`, `vital_thresholds`, `spo2_bias_guard`. `backend/model/` holds the risk model and its artifacts.

4.3 Two ASGI applications, not one

Knowing which app is running matters when reading logs. Both are live, and booting only one and declaring the backend verified is a mistake that has already been made once here.

App	Module and surface
Orchestrator	<code>triage/orchestrator/app.py</code> — <code>/v1/*</code> , 20 routes. This is what the web app calls. A lifespan hook bootstraps a <code>World</code> and starts a tick loop
Risk-model API	<code>triage/api.py</code> — <code>/score</code> , <code>/override</code> , <code>/queue</code> , <code>/recheck/{id}</code> , <code>/surge-status</code> , <code>/audit/{id}</code> , <code>/demo/arrival</code>

The orchestrator's twenty routes group as: census and state (`/config`, `/census`, `/encounter/{id}`, `/stream`); scoring (`/score`, `/decision`, `/rechecks`); intake (`/intake/submit` and five `/intake/tree/*` routes); perception (`/speech/transcribe`, `/structure`); measurement (`/encounter/{id}/measurement`); surge and audit; and two demo-control routes. Note that `/override` on the risk-model app is a first-class route rather than an administrative afterthought — a human reversal is a normal, recorded event here, not an exception path.

4.4 Config is the clinical surface

Six YAML files under `backend/config/` hold behaviour that would otherwise be buried in Python:

File	What it governs
<code>age_strata.yaml</code>	Six strata with per-stratum normal vital ranges
<code>band_cadence.yaml</code>	Re-score and re-measure intervals, wait ceilings, breach actions, per band
<code>red_flags.yaml</code>	Observation → Red mapping, with per-flag override authority
<code>surge_policy.yaml</code>	Detection window, 3× threshold, what may stretch
<code>label_spec.yaml</code>	The training label; frozen and hashed into the manifest
<code>feature_registry.yaml</code>	Every field classified SAFE / CONDITIONAL / PROHIBITED

The registry is the enforcement artifact behind the white paper's leakage-lint claim, and it **fails closed**: `model/feature_registry.py` treats any unregistered name as `PROHIBITED`, so adding a generator field without classifying it breaks the build. That is the point — it is how feature drift gets caught mechanically instead of in review.

4.5 Runtime workflow, end to end

1 • Intake. Ten steps driven by `web/intake/IntakeApp.tsx`: Welcome → Companion → HumanOffer → Consent → Basics → Conversation → Pain → Readback → Token, with a `HumanLane` bail-out available at any point. Conversation walks one of sixteen branches in `web/intake/tree/branches/` (chestPain, breathing, trauma, obstetric, paed, poisoning, ...). Branch choice is a local keyword table first; Groq refines it only if the patient has not yet answered inside the branch. The backend mirrors this for the live path in `backend/intake/pipeline.py`, which makes *no* clinical decision beyond the deterministic red-flag table.

2 • Scoring. `backend/model/risk_model.py` runs a fixed nine-stage pipeline: stratum resolution → vital freshness → deterministic red flags (which fire first) → vital thresholds → raw risk → per-stratum calibration → asymmetric reliability weighting → conformal uncertainty → a `ScoreObject` or an `AbstentionObject`, never a partial object. Then `triage/band_engine.py` assigns the band, and the `SpO2` bias guard runs before any de-escalation attempt is even considered.

3 • The waiting loop (Fig. 2). `triage/recheck_scheduler.py` re-scores everyone every five minutes and re-measures on the band cadence; stale readings pull a re-measurement forward and ceiling breaches escalate. A Green patient who misses two consecutive rechecks goes Yellow. An abstained patient holds at Yellow and breaches at fifteen minutes unreviewed. `triage/surge_controller.py` detects 3× normal arrival rate and stretches Yellow 30→45 and Green 60→90 minutes. Red never stretches.

4 • The human closes it. Override is one touch from every surface showing a band. `triage/audit_log.py` appends a tamper-evident record rendered verbatim on `/audit`, and consent withdrawal is a new event rather than a deletion.

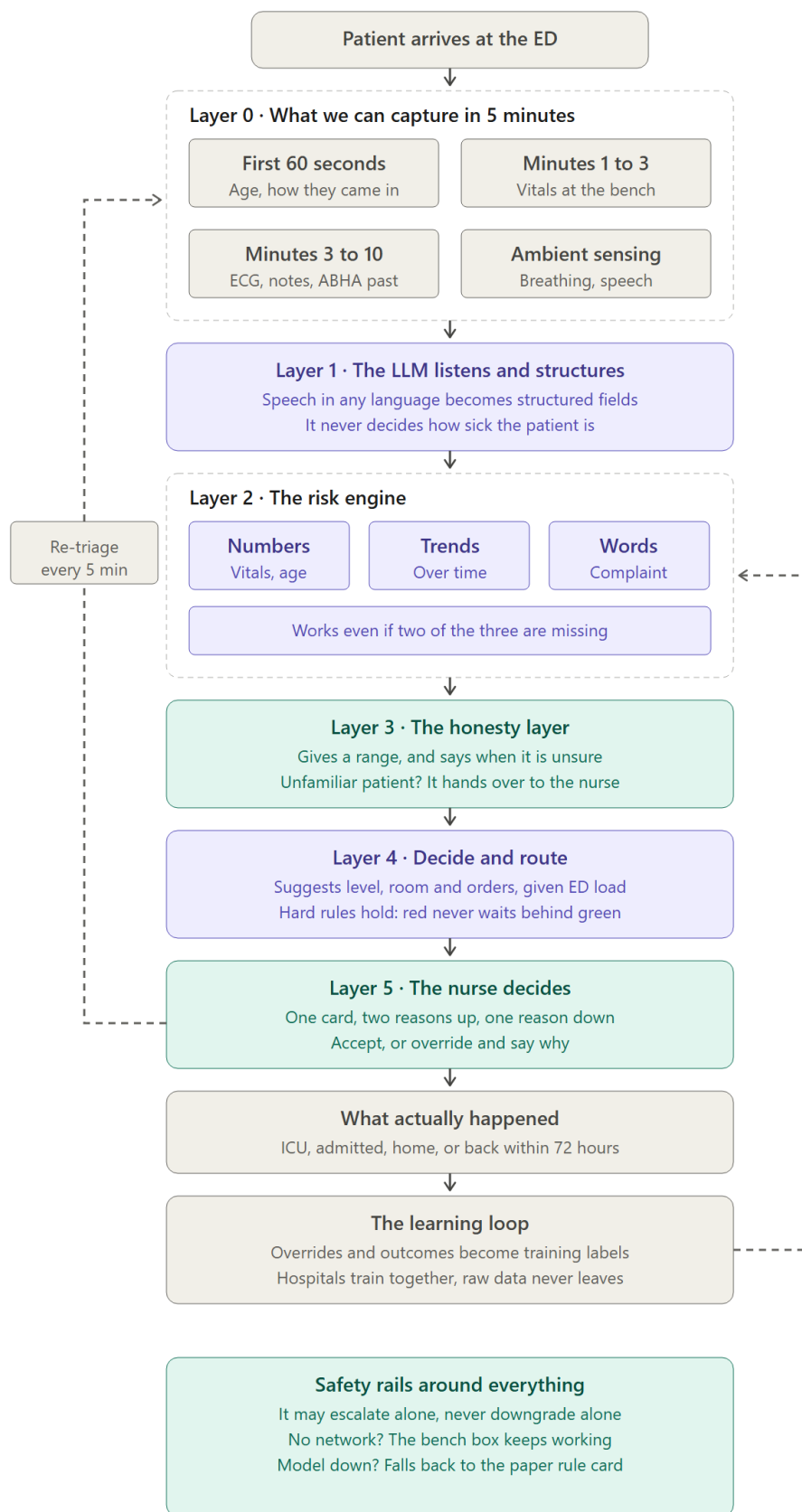


Figure 1. The six-layer pipeline across three tracks. Track A (perception) may only report; Track B (interfaces) may only render; Track C (the risk engine) is the only track permitted to assign a band. The dashed returns are the two feedback loops: re-triage every five minutes, and overrides plus outcomes becoming training labels.

outcome_ref is back-filled later — that is the training signal.

4.6 Degradation, and what the model may not do

The LLM is confined to Track A (Fig. 3). It is a structural parser: unstructured narrative in, a rigid schema out. It is prohibited from producing an acuity score, assigning a band or issuing a clinical judgement, and the prohibition is structural rather than advisory — no production rule for acuity exists in its output grammar, so the sentence cannot be formed. Tier-2 option matching is the sharpest illustration: the model is handed the options already on the patient's screen and asked to return one of their values or NONE, and anything else it returns is read as NONE on both sides of the wire.

Degradation is symmetrical on the model side. If the artifact is missing, its sklearn version mismatches, or its feature contract has drifted from the live extractor, `model/artifact.py` falls back to the frozen AIIMS-ATP rule card and stamps `score_source="fallback_heuristic"`. It does not raise into the triage API. The UI never shows a blank state.

5 SYNTHETIC DATA GENERATION

No patient health information exists anywhere in this repository. Everything is manufactured by `backend/data/generator/`, in four layers, and the design of the label layer is the part worth reading.

5.1 Conditions — joint, not independent, vitals

`conditions.py` defines clinical archetypes, each with a *correlated joint* distribution over vitals per age stratum. Vitals are never sampled independently, because independent sampling mints physiologically impossible patients. Each vital is a truncated normal with hard clipping bounds.

5.2 Trajectories — 37 timesteps per patient

`trajectories.py` builds a vital-sign trajectory from T=0 to T=180 minutes at 5-minute intervals, in one of three shapes: *compensating-then-decompensating* (holds, then deteriorates), *improving*, and *stable-sudden* (stable, then rapid deterioration at a random inflection point). Seven vitals are tracked — hr, rr, bp_sys, spo2, temp_c, gcs, pain_score — and each measurement carries the (value, timestamp, source, validity) tuple the freshness model requires.

5.3 Missingness — three mechanisms, plus device bias

`missingness.py` does not apply uniform random dropout, because that is not how data goes missing in an ED. Three distinct mechanisms are modelled: *sensor failure* (reading present but flagged invalid), *zero history* (a first-visit patient with no prior records), and *staff shortage* (reading simply absent, nobody had time). The same module applies a calibrated upward SpO₂ offset for the darker-skin flag, modelling the occult hypoxaemia under-detection that the bias guard downstream exists to catch.

5.4 Labels — and why they are not circular

`labels.py` derives the outcome from the per-patient latent, never from the assigned band — training on band labels would encode their ~32% mistriage rate. But the naive alternative,

label = $f(\text{severity})$, produces a fake result: measured on this generator, late-window peak severity is essentially a lookup on trajectory shape, and a model trained on it scores AUROC 0.988 from T=0 vitals alone because it is really doing condition identification. Three structural choices prevent that:

1. **The label lives strictly in the future.** Features come from steps $\leq k$; the label is computed over $(k, \text{end}]$. Because the inflection point is per-patient and random, the future is genuinely underdetermined by the observation window.
2. **A frailty latent enters the outcome and never touches any vital.** No feature set can recover it, which puts a hard floor under Bayes error.
3. **The outcome is a Bernoulli draw, not a threshold.** Two patients with identical conditions and near-identical vitals can differ in outcome.

Together these bound the achievable AUC well below 1.0, and `severity_oracle_auc()` computes that ceiling explicitly so the trained model can be reported against it rather than against perfection. The primary head is `critical_composite_h180`: the horizon is 180 minutes, not 24 hours, and it is named that way deliberately — calling it "24h" would be a false claim about what was measured on synthetic data. `config/label_spec.yaml` is frozen and hashed into the artifact manifest, so changing any value there invalidates every trained artifact.

5.5 Honest stratum coverage

Five conditions define vitals only for adult, and `vitals_for_stratum()` silently falls back to the adult distribution for anything else. Sampling (condition $\times$ stratum) freely would therefore mint neonates carrying adult trauma physiology, and any per-stratum fairness metric computed over those rows would be measuring nonsense. Sampling is restricted to explicitly-defined pairs, and every row records `stratum_is_fallback` for audit — coverage that is narrower and honest rather than broad and fabricated.

5.6 The two datasets

`corpus.py` builds the fixed 20-record demo corpus P-01...P-20 at seed 42 (P-01...P-10 map to the §10 acceptance cases in the brief; P-11...P-20 add strata and conditions for generalisation). `bulk.py` builds the N-parameterised training population. A further `dynamic_encounters.json` adds twelve arrivals that appear over the course of a demonstration, which is why the census returns 32 and not 20.

6 MODEL TRAINING

`model/train.py` runs nine stages and writes a versioned artifact:

1. Load the labelled dataset from `bulk.py`
2. Build features through `model/features.py` — the shared extractor, also used at serving time
3. Split by `patient_id`, stratified on (stratum, label)
4. Auxiliary regressor → out-of-fold derangement prediction
5. Primary GBDT classifier
6. Per-stratum isotonic calibration, on `calib_iso`



Figure 2. The four lanes out of the triage bench, and what each one guarantees. The abstention lane ("not sure") is a first-class outcome: no colour is offered, the queue is never auto-sorted, and a human decides.

7. Mondrian split-conformal quantiles, on `calib_conf`
8. Cost-ratio thresholds solved on the calibration split
9. Persist everything plus a manifest

6.1 Two heads, honestly described

sklearn has no multi-task GBDT, and lightgbm/xgboost were unavailable while torch was broken in this environment. So this is a two-stage *stack*, not multi-task learning over a shared trunk: the auxiliary regressor's out-of-fold prediction becomes one input column to the primary classifier. That achieves the white paper's "secondary label constrains the primary" effect. It is not the same architecture, and the code says so in its own docstring.

6.2 Split discipline

`calib_iso` and `calib_conf` are **separate splits**. Fitting isotonic regression and then computing conformal quantiles on the same rows makes those scores in-sample, breaks exchangeability and voids the coverage guarantee — while still printing coverage numbers that look entirely fine. That is the standard way this goes silently wrong, and the separation is the defence.

6.3 Features

59 features from `model/features.py`. Per vital, six: the value, a per-stratum z -score, the reading's age in minutes, a slope per hour, a 30-minute delta, and a count of readings. That shape is what makes deterioration-while-waiting visible to a tabular model at all — a snapshot model cannot see a

trend.

6.4 The sequence track

`model/train_seq.py` replicates the same calibration, conformal and threshold pipeline over PyTorch sequence models — a last-observation baseline, GRU, TCN and Transformer — reusing `train.py`'s split and calibration functions directly rather than reimplementing them. This is the experimental track; it is not what ships.

7 EVALUATION

Evaluation is split in two: the risk model (`model/evaluate.py`, `model/leaderboard.py`) and the perception layer (`eval/`). Both report numbers rather than adjectives, and both were designed to make self-congratulation mechanically detectable.

7.1 Every metric, three ways

`model/evaluate.py` reports each headline metric for the trained model, for the hand-coded baseline it must beat, and for a *severity oracle* that knows the true future peak severity. The oracle row is the anti-self-congratulation control: a trained model that approaches or exceeds it has leakage, and that is a mechanical detection rather than a judgement call.

The metric choices follow the white paper rather than convenience. AUPRC is primary, because AUROC flatters every model at low prevalence. The safety metric is **under-triage at a fixed over-triage rate**, not accuracy. ECE

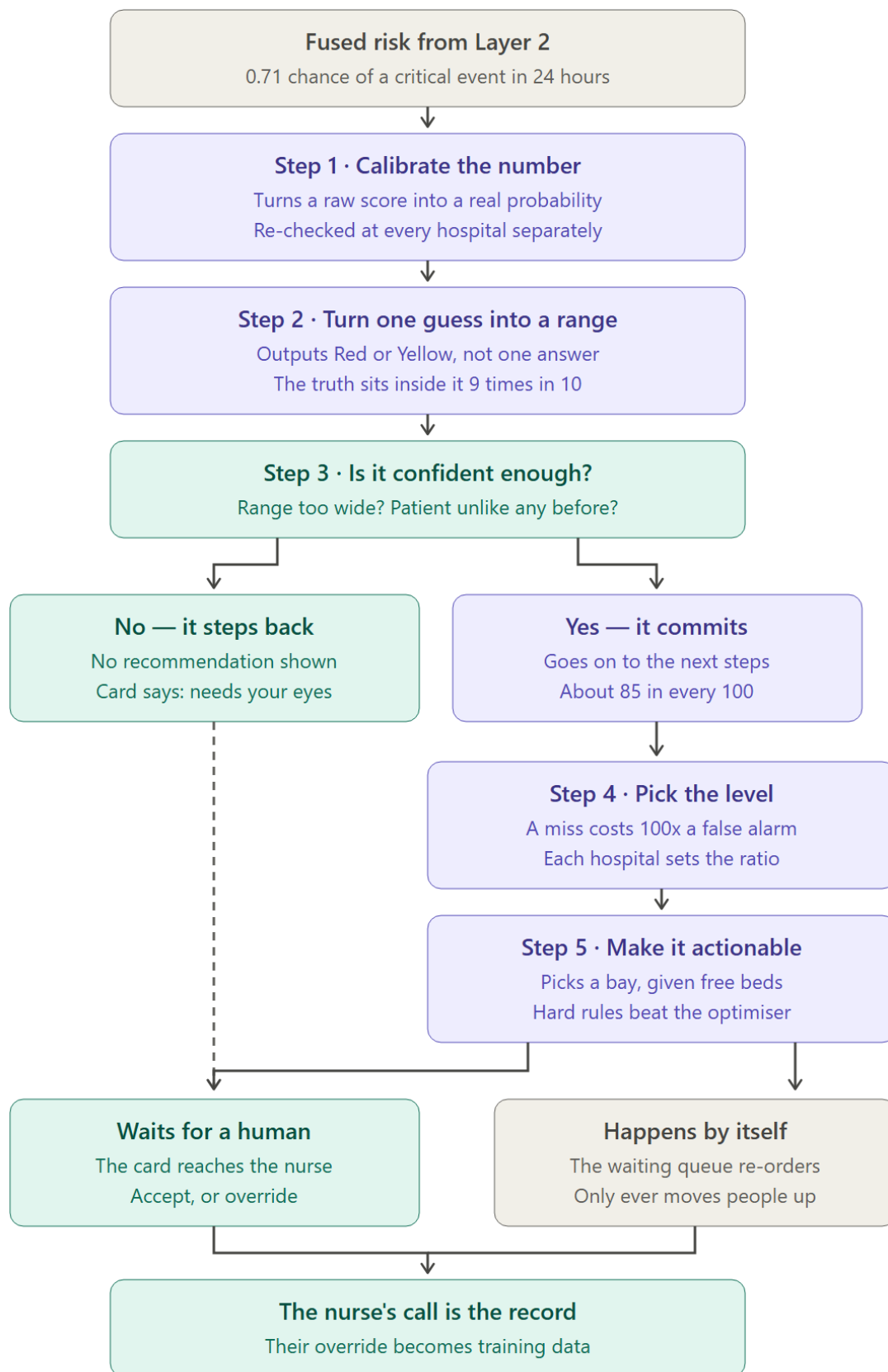


Figure 3. The post-engine decision path. Calibration, then a range rather than a point, then an explicit confidence gate that may step back entirely; only past that gate does the system pick a level, and even then the nurse's call is the record.

uses *quantile* bins, because at $\sim 10\%$ prevalence equal-width bins put nearly all mass in bin 0 and ECE collapses to ≈ 0 regardless of real calibration quality. Fairness is equalised false-negative rate across strata with bootstrap CIs, never a bare max–min gap, which at these positive counts would be mostly noise.

7.2 The shipped artifact's committed numbers

Over $n = 4001$ held-out encounters at prevalence 9.8%:

	AUROC	AUPRC
Trained model	0.689	0.174
Hand-coded baseline	0.630	0.156
Severity oracle (ceiling)	0.866	0.410

Conformal coverage is 0.910 overall with a mean set size of 1.08; overall ECE is 0.011; per-stratum FNR runs from 0.00 (adolescent, $n_{\text{pos}} = 35$) to 0.15 (adult, $n_{\text{pos}} = 164$), each with bootstrap CIs and an underpowered flag. All eight `go_live_criteria` in `metrics.json` currently evaluate true. Under-triage at a fixed 30% over-triage rate is 0.486 — these are modest numbers on a synthetic task with a deliberate Bayes floor, and they are reported as they stand.

7.3 The standing rule for model results

`model/leaderboard.py` exists to enforce one rule: no model result is reported, discussed or shipped in isolation. Every model runs through the harness and lands as one row beside the shipped baseline, the hand-coded rule card, the prevalence floor and the oracle ceiling. *A number quoted without its row in that table is not a result yet.*

7.4 Perception bake-offs

`eval/` answers two questions with numbers. Which Whisper should transcribe the intake conversation (`run_asr_bakeoff.py`, against `asr_manifest.json`), and which LLM should extract the structured fields while the perception-only guarantee holds (`run_structurer_bakeoff.py`, against `structurer_cases.json`, which is deliberately messy: adversarial, ambiguous, negation, Hinglish, prompt-injection, obstetric, paediatric and reliability cases). Both import scoring from `eval/metrics.py`, so a number in a notebook and a number in the report were produced by the same code. Results land in `docs/benchmarks/`. The structurer table also records `schema_failures` and `forbidden_key_hits` — the latter is how the perception-only boundary is measured rather than assumed.

Both bake-offs run locally against hosted models and on a Kaggle T4 against open-weight ones, because the development laptop is an AMD Radeon RX 6500M with no CUDA and no ROCm-on-Windows. **T4s are Turing and do not support bfloat16**; every notebook uses float16 or 4-bit NF4, and a model card that says `torch_dtype=torch.bfloat16` will crash there.

7.5 The invariant suite

282 tests collected across `tests/` and `intake/`, covering age stratification (10), the audit log (6), the band engine (7), corpus acceptance cases (10), features (12), the invariants themselves (15), model training (11), the orchestrator contract (12), reliability weighting (6), surge control (8), and the intake

pipeline, conversation flow, tree branching, classifier and voice normalisation (71 across six files).

8 USING THIS REPOSITORY

Run Python from `backend/`, not from the repository root. `intake`, `model`, `trriage`, `data`, `rules`, `speech` and `eval` are top-level absolute import roots, and `confest.py` puts that directory on `sys.path`. Both `pytest` and `uvicorn` must be started from there.

8.1 Mock mode — the zero-dependency demonstration

This is the default and the insurance policy. It needs Node and nothing else, and it is the demo path.

```
cd web
npm install
cp .env.example .env.local
# NEXT_PUBLIC_MP_SOURCE=mock is already the default
npm run dev
```

Open `http://localhost:3000`. All seven surfaces work, driven by the twenty-patient corpus in the browser. Use Chrome if you intend to speak to the kiosk: `webkitSpeechRecognition` is Chrome-only, and every question stays answerable by tap or keyboard so a denied microphone degrades the experience rather than blocking intake.

8.2 Live mode — the full pipeline

```
cd backend
python -m venv .venv
# Windows: .\venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
python -m uvicorn triage.orchestrator.app:app \
--port 8000 --reload
```

The second application runs beside it, on its own port:

```
python -m uvicorn triage.api:app --port 8001 --reload
```

Then point the frontend at the orchestrator: set `NEXT_PUBLIC_MP_SOURCE=live` in `web/.env.local` and restart `npm run dev`. The variable is read at build time, so the restart is not optional — editing the file alone changes nothing.

For the fully local perception path:

```
pip install -r requirements-speech-local.txt
set MEDIPILOT_ASR_BACKEND=local,groq
set MEDIPILOT_STRUCTURER=local,groq,rules
python -m uvicorn triage.orchestrator.app:app --port 8000
```

8.3 The test suite

```
cd backend
python -m pytest
```

Expect **278 passed, 4 failed** (§10). `pytest.ini` sets `testpaths = tests intake`; `speech/` is deliberately excluded because `speech/test_whisper.py` needs `torch`, `ffmpeg` and downloads model weights, which would break a default run for anyone on the hosted path. Run it explicitly with `pytest speech/`.

8.4 Regenerating data and artifacts

The training set and the held-out split are gitignored as regenerable, and they are what three of the four failing tests

want:

```
cd backend
python -m data.generator.bulk --n 20000 \
--out data/train_set.jsonl
python -m model.train --data data/train_set.jsonl
python -m model.evaluate
python -m model.leaderboard
```

Seeds default to 1337, so runs are reproducible. To rebuild the fixed demo corpus instead, `python -m data.generator.corpus (seed 42)`. The perception bake-offs are separate entry points — `python -m eval.run_asr_bakeoff` and `python -m eval.run_structurer_bakeoff` — and both skip their hosted candidates with a printed notice when `GROQ_API_KEY` is absent rather than failing.

8.5 Production execution

```
cd web && npm run build && npm start
```

Do not add workers to either Python app as it stands. The orchestrator holds `world = World()` at module level and starts a tick loop in its lifespan hook; `triage/api.py` likewise holds `_audit_log`, `_scheduler`, `_surge_ctrl` and `_surge_state` at module level. Running `gunicorn --workers 4` would create four independent worlds with four independent tick loops, and the census a nurse sees would depend on which worker answered. Until that state is externalised (§9), the correct production posture is one process per app behind a reverse proxy.

8.6 Which adapter is running?

`web/lib/api/client.ts` picks its adapter from `NEXT_PUBLIC_MP_SOURCE`. Before debugging any "the backend is wrong" report, check which one was in play: **a mock-mode screenshot proves nothing about the service.**

Variable	Read by	Purpose
<code>NEXT_PUBLIC_MP_SOURCE</code>	web	mock (default) or live
<code>NEXT_PUBLIC_API_BASE</code>	web	orchestrator URL; default port 8000
<code>GROQ_API_KEY</code>	web server, backend	optional; unlocks the hosted tiers only
<code>MEDIPILOT_INTAKE_OFFLINE</code>	web server	1 forces the offline path
<code>MEDIPILOT_STRUCTURER</code>	backend	fallback order, e.g. local, groq, rules
<code>MEDIPILOT_ASR_BACKEND</code>	backend	fallback order, e.g. local, groq

`GROQ_API_KEY` deliberately carries no `NEXT_PUBLIC_` prefix, so it is never inlined into a browser bundle — it is read exclusively by the four route handlers under `web/app/api/intake/`. **Never add the prefix.**

8.7 Privacy and the audit ledger

With `GROQ_API_KEY` omitted the system runs entirely on-premise: local ASR, offline matching, the deterministic structurer, and no outbound request on any path. Every autonomous decision and every clinical override is appended to the hash-chained ledger in `triage/audit_log.py`, inspectable at `/audit`. Each `OverrideRecord` carries 17 substantive fields — including `inputs_hash`, so the decision is reproducible, and `factors_shown`, so the card as displayed is preserved — plus `record_hash` and

`previous_record_hash`, 19 in total. The chain makes a retroactive edit *detectable*; it does not make one impossible.

9 SCALING: THE TARGET STACK

Everything above describes what runs today: two single-process Python apps and a Next.js payload on one machine, with all state in memory. That is the right shape for a prototype and the wrong shape for a hospital. Figure 4 is the target, and this section is the honest gap between them.

Layer	Today	At scale
State	in-process per app	World, Postgres for encounters + Redis for the live queue; this is the <i>blocking</i> change
Serving	one uvicorn process	gunicorn/uvicorn workers, then replicas behind a load balancer — only after state moves out
Inference	sklearn joblib, in-process	LightGBM → ONNX Runtime on the edge box; millisecond latency, no Python at the bench
Local store	none	SQLite on the edge box, so the bench survives a network loss
Ticks	asyncio loop in the app	Kafka event stream on 5-minute ticks; the loop becomes a consumer, not a singleton
Client	Next.js dev/start	the same PWA build on board and phone; installable, offline-capable
EHR	none	FHIR resources over CDS Hooks, so the card appears inside the existing workflow
Features	computed inline	Feast + Redis, so training and serving read one definition
Training	a script on a laptop	Argo pipelines with MLflow tracking in the regional cloud
Structurer	hosted Groq	a self-hosted 8B under the same output grammar, on the regional node
Monitoring	metrics.json train time	at drift watchdogs on feature and outcome distributions, alarming per site
Multi-site	single machine	weekly federated rounds with secure aggregation

Three observations about that table. First, the ordering is not free: **externalising state is the prerequisite for every horizontal move**, and attempting workers first produces a system that looks scaled and silently disagrees with itself. Second, the edge tier is what makes the offline guarantee real at more than one site — ONNX plus SQLite plus a PWA is a bench that keeps working with the network cut, which is the same property the mock adapter gives the demo today. Third, the federated row is the only one that is a *commitment* rather than an optimisation: raw patient data never leaves the building, so cross-site learning has to be federated or it does not happen at all.

None of this is implemented. It is drawn, costed at a target of under Rs. 5 per encounter in the white paper, and

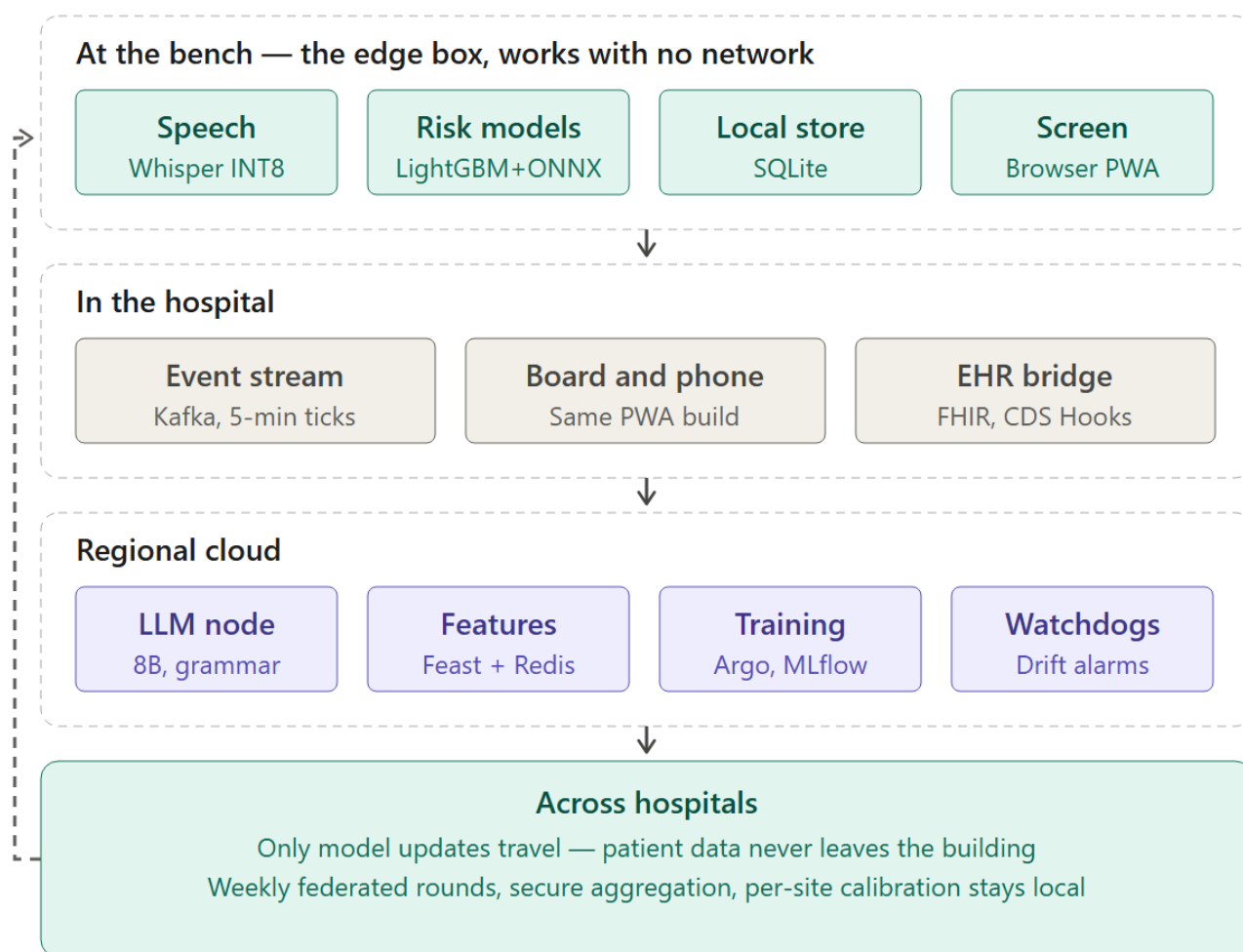


Figure 4. The target deployment tiers. The bottom row is the invariant: only model updates travel between sites, and per-site calibration stays local.

deliberately not started — the prototype’s job was to establish the clinical contract, and a distributed system would have made that harder to verify, not easier.

10 VERIFICATION STATE AND KNOWN ISSUES

Reported as measured, not as intended. `python -m pytest` from `backend/` collects 282 tests and gives **278 passed, 4 failed** in roughly eight seconds. The counts are identical before and after the restructure, so these are pre-existing failures and not migration damage. None is a failing safety invariant.

Failing test	Cause and remedy
<code>test_auroc_leakage_canary</code>	Needs <code>test_split.npz</code> , gitignored as regenerable. Restore by retraining (§8)
<code>test_auprc_beats_prevalence_baseline</code>	The same missing held-out split
<code>test_conformal_coverage_at_least_090</code>	The same missing held-out split
<code>test_census_has_20</code>	Asserts a census of 20; it is now 32 (20 seeded + 12 dynamic arrivals). The assertion is stale, not the code

The distinction matters. The first three are a *packaging* decision — the training set and its held-out split are reproducible from a seed, so they are generated rather than

committed, and the tests that consume them cannot run until they are. The fourth is an ordinary stale assertion left behind when dynamic arrivals were added: true when written, not revisited. Both are stated here, because a README reporting 278 passing tests while omitting four failures is a README that has to be re-verified by whoever trusts it.

10.1 Other known issues, stated plainly

- `npm run lint` hangs with no output and has to be killed. It did this before the restructure too. Lint is currently not a usable gate.
- The orchestrator OpenAPI *baseline* capture is empty — 0 paths recorded against 20 in the after-state. That before/after comparison did not actually run; do not cite it as evidence. The after-state capture is real and is what §4 draws on.
- `web/README.md` is still stock `create-next-app` boilerplate.
- The root `README.md` predates the restructure and still describes the old tree in places, including at least one `uvicorn` target that is not a real module path.
- Neither Python app is safe to run multi-worker (§8).
- The audit-record field count is documented inconsistently elsewhere — `audit_log.py` says 15, the root README says 16. Counted from the dataclass: 17 substantive fields plus 2 chain fields.

What to check before believing a run. The frontend inlines `NEXT_PUBLIC_` variables at *build* time — if a change to `.env.local` appears to do nothing, the dev server was not restarted. The two ASGI apps answer on different route sets, so a 404 on `/v1/census` usually means `triage/api.py` is running instead of the orchestrator. And a screenshot taken in mock mode is evidence about the interface only.

MediPilot • “Nobody waits unseen.”

COMMITMENTS AT A GLANCE

Commitment	Where it is enforced in the code
No LLM acuity	No production rule for acuity in the output grammar; <code>intake/llm_structurer.py</code>
No autonomous de-escalation	<code>AsymmetricAutonomyViolation</code> raised on the code path, not logged
No silent guess	Conformal gate; <code>AcuityCard</code> throws without a confidence band
No hard model dependency	A deterministic bottom tier on every ladder, down to the frozen ATP rule card
No leaked feature	<code>config/feature_registry.yaml</code> fails closed on any unregistered field
No circular label	Future-window label, unobservable frailty label, Bernoulli outcome
No isolated result	<code>model/leaderboard.py</code> : a number without its row is not a result
No PHI anywhere	Synthetic corpus only; on-premise by default when the key is absent
No unverifiable claim	<code>.migration/</code> carries build, lint, routes, OpenAPI and the test run

Note on format. This README follows the house style of the Round I white paper ([docs/submission/BIT01_solution.pdf](#)) so that the submission reads as one document set. It is generated from `docs/paper/implementation_readme.tex` and was audited against the tree on branch restructure; the prose source of record for the same material is `docs/README.md`, and the system-level map is the `medipilot` skill.